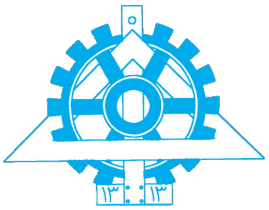
University of Tehran
College of Engineering
School of Electrical and Computer Engineering



Machine Learning

Assignment #5

Student Name
Amirali Dehghani

Student ID
810102443

February 6, 2026

Contents

List of Figures

List of Tables

Abstract

In this assignment, we explored fundamental concepts of machine learning, ranging from classical regression techniques to the theoretical foundations of deep neural networks. The study is divided into experimental and theoretical analyses:

Polynomial Regression and Regularization: We investigated the bias-variance trade-off using polynomial regression on synthetic datasets. Visual and numerical analysis confirmed that lower-degree models (e.g., $d = 1, 3$) suffered from underfitting, while high-degree models (e.g., $d = 12$) exhibited severe overfitting. We demonstrated that applying **Ridge Regularization** (ℓ_2 penalty) effectively constrains the model complexity, significantly reducing the validation MSE for the high-degree model by identifying an optimal regularization parameter λ.

Neural Network Representation: We analyzed the expressivity of computation graphs. By mapping mathematical functions to specific network architectures (linear and ReLU-based), we manually derived the optimal weights and biases required to approximate piecewise linear and non-linear functions.

Loss Functions and Probabilistic Interpretations: We mathematically proved the equivalence between Maximum Likelihood Estimation (MLE) and minimizing Cross-Entropy error. This analysis was extended to scenarios involving label noise (deriving a robust loss function) and alternative target representations ($\{-1, 1\}$), identifying tanh as the appropriate activation function for the latter.

Optimization Stability and Initialization: Finally, we addressed the challenges of training deep networks. We provided a statistical

proof for the **"Dead ReLU"** phenomenon, showing how improper bias initialization leads to vanishing gradients and permanently inactive neurons. Furthermore, we derived the **Xavier (Glorot) Initialization** method by enforcing variance preservation constraints across layers. Numerical analysis highlighted that minor initialization errors lead to exponential variance growth in deep networks, posing significant stability risks, particularly in half-precision (FP16) training.

Question 1: Maximum Likelihood Estimation

Given the distribution $P(x|\theta) = \theta x^{-\theta-1}$:

$$L(\theta) = \prod_{i=1}^{n} P(x_i|\theta) = \prod_{i=1}^{n} \theta x_i^{-\theta-1} = \theta^n \left(\prod_{i=1}^{n} x_i \right)^{-\theta-1}$$

$$\xrightarrow{\ln(\cdot)} \ell(\theta) = n \ln \theta - (\theta + 1) \sum_{i=1}^{n} \ln x_i$$

$$\xrightarrow{\frac{\partial}{\partial \theta} = 0} \frac{n}{\theta} - \sum_{i=1}^{n} \ln x_i = 0$$

$$\Rightarrow \frac{n}{\hat{\theta}} = \sum_{i=1}^{n} \ln x_i$$

$$\Rightarrow \boxed{\hat{\theta}_{\text{MLE}} = \frac{n}{\sum_{i=1}^{n} \ln x_i}}$$

Question 2: Logistic Regression with One Feature

$$s(\gamma) = \frac{1}{1+e^{-\gamma}} \quad \xrightarrow{\frac{d}{d\gamma}} \quad s'(\gamma) = s(\gamma)(1-s(\gamma))$$

$$\frac{\partial s}{\partial \alpha} = s(1-s)\underbrace{\frac{\partial(x-\alpha)}{\partial \alpha}}_{-1} = -s(1-s)$$

$$J(\alpha) = -y \ln s - (1-y)\ln(1-s)$$

$$\xrightarrow{\frac{\partial}{\partial s}} \frac{\partial J}{\partial s} = \frac{-y(1-s)+s(1-y)}{s(1-s)} = \frac{s-y}{s(1-s)}$$

$$\xrightarrow{\text{Chain Rule}} \frac{\partial J}{\partial \alpha} = \underbrace{\frac{\partial J}{\partial s}}_{\frac{s-y}{s(1-s)}} \cdot \underbrace{\frac{\partial s}{\partial \alpha}}_{-s(1-s)}$$

$$\rightarrow \frac{\partial J}{\partial \alpha} = -(s-y) = y-s$$

$$\alpha_{new} \leftarrow \alpha - \epsilon \frac{\partial J}{\partial \alpha}$$

$$\Rightarrow \boxed{\alpha_{new} \leftarrow \alpha - \epsilon(y-s)}$$

Question 3: L2-Regularized Linear Regression with Newton's Method

$$J(w) = (Xw - y)^\top (Xw - y) + \lambda w^\top w$$

$$\xrightarrow{\nabla} \nabla J(w) = 2(X^\top X + \lambda I)w - 2X^\top y$$

$$\xrightarrow{\nabla} H = \nabla^2 J(w) = 2(X^\top X + \lambda I)$$

$$w_1 = w_0 - H^{-1}\nabla J(w_0)$$

$$\rightarrow w_1 = w_0 - \underbrace{[2(X^\top X + \lambda I)]^{-1}}_{\frac{1}{2}H^{-1}}[2(X^\top X + \lambda I)w_0 - 2X^\top y]$$

$$\rightarrow w_1 = w_0 - \left(\underbrace{(X^\top X + \lambda I)^{-1}(X^\top X + \lambda I)}_{=I} w_0 - \underbrace{(X^\top X + \lambda I)^{-1} X^\top y}_{=w^*} \right)$$

$$\rightarrow w_1 = w_0 - (Iw_0 - w^*)$$

$$\Rightarrow w_1 = w_0 - w_0 + w^*$$

$$\Rightarrow \boxed{w_1 = w^*} \quad \checkmark$$

Question 4: Polynomial Regression

4.1 Model Fitting on Dataset A

We fitted polynomial regression models with degrees $d \in \{1, 3, 9\}$ to the training set `data_A.csv`. The resulting fitted curves alongside the training data are shown in Figure 1.

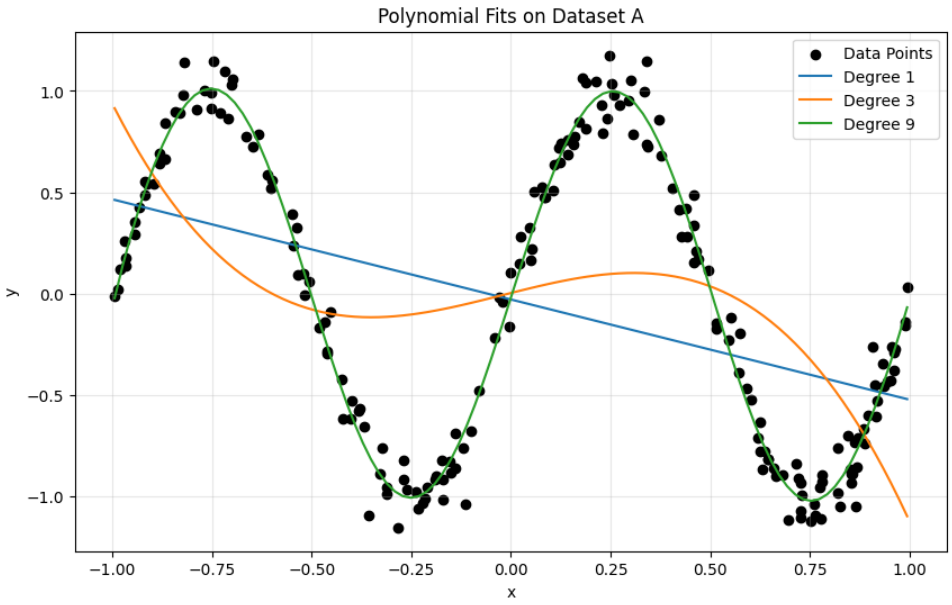


Figure 1: Polynomial regression fits for degrees 1, 3, and 9 on Dataset A.

Analysis of Results:

- **Degree 1 (Linear):** The model learns a straight line which clearly fails to capture the oscillating, non-linear pattern of the data. This is a classic case of **underfitting** (high bias).
- **Degree 3 (Cubic):** While it introduces some curvature, it is still insufficient to model the multiple peaks and troughs present in the data. It remains an underfitted model.

- **Degree 9:** This model fits the data very well, closely following the sinusoidal shape of the underlying function.

  Overfitting Check: Visually, the degree 9 curve is smooth and does not exhibit high-frequency oscillations or "wiggles" between data points that are characteristic of overfitting. It appears to capture the true complexity of the signal rather than the noise. Therefore, degree 9 is the most appropriate model among the chosen options.

4.2 Effect of Polynomial Degree on Generalization

We evaluated polynomial regression models with degrees $d \in \{1, \ldots, 12\}$ on Dataset B. The Mean Squared Error (MSE) for both training and validation sets was computed for each degree to identify the bias-variance trade-off. The results are shown in Figure 2.

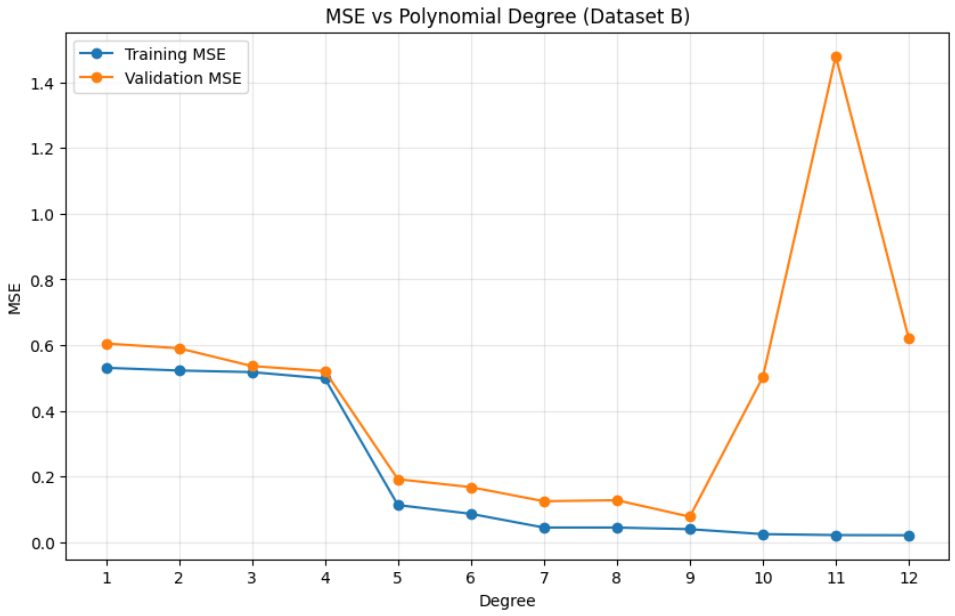


Figure 2: Training and validation MSE as a function of polynomial degree for Dataset B.

Analysis of Results:

- **Underfitting ($d = 1$ to 4):** For lower degrees, both training and validation errors remain high (MSE $\approx 0.5 - 0.6$). The model lacks sufficient complexity to capture the data's structure, resulting in high bias.
- **Optimal Complexity ($d = 9$):** As the degree increases from 5 to 9, the validation error decreases significantly. The minimum validation MSE is observed at **Degree 9** ($\text{MSE}_{val} \approx 0.0771$), which represents the best trade-off between bias and variance for this dataset.
- **Overfitting ($d = 10$ to 12):** Beyond degree 9, while the training MSE continues to drop (reaching ≈ 0.02 at $d = 12$), the validation MSE spikes dramatically (e.g., reaching ≈ 1.48 at $d = 11$). This large divergence indicates the model is overfitting the noise in the training set and failing to generalize (high variance).

4.3 Effect of Ridge Regularization

To mitigate the overfitting observed with high-degree polynomials, we applied Ridge Regression to the model with degree $d = 12$. We varied the regularization parameter λ across a logarithmic scale from 10^{-6} to 10^{2}. The evolution of Training and Validation MSE is depicted in Figure 3.

Analysis of Results:

- **Small λ (Overfitting Regime):** For very small values (e.g., $\lambda < 10^{-5}$), the regularization term is negligible. The model behaves like standard least squares, resulting in low training error but

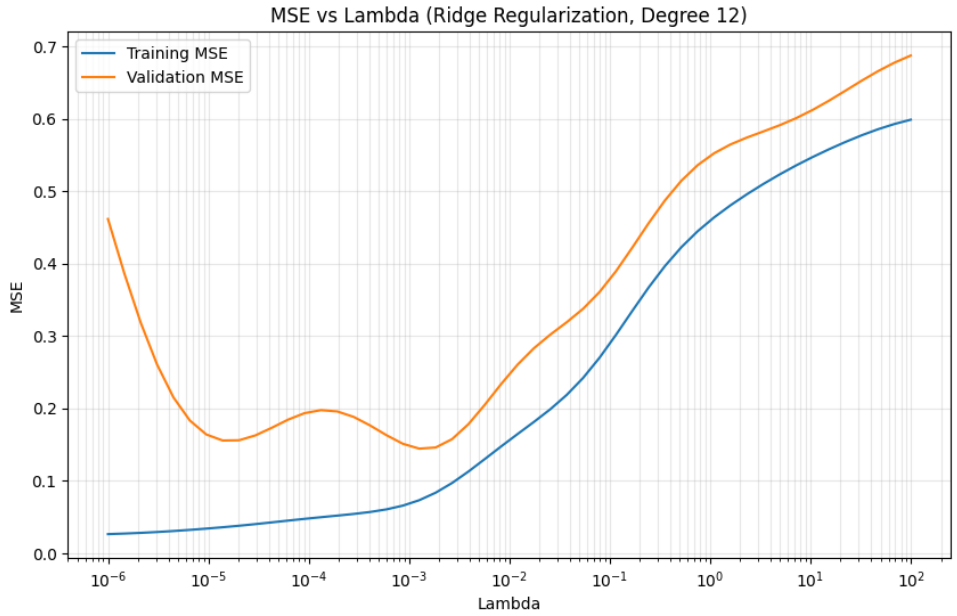


Figure 3: Training and validation MSE vs. regularization parameter λ for a degree 12 model.

relatively high validation error (MSE ≈ 0.45), confirming that the degree 12 model is still overfitting.

- **Optimal λ (Generalization Sweet Spot):** As λ increases, the penalty on the weights restricts the model's flexibility, reducing variance. The validation error drops significantly, reaching a minimum of **0.1444** at $\lambda \approx$ **0.00127**. This demonstrates that regularization successfully "tamed" the complex degree 12 model to generalize better.

- **Large λ (Underfitting Regime):** As λ grows beyond the optimal point (e.g., $\lambda > 0.1$), the penalty becomes too severe, forcing the weights towards zero. This restricts the model too much, causing both training and validation errors to rise sharply, a sign of underfitting (high bias).

Conclusion on Polynomial Regression: Comparing the results, the unregularized degree 12 model had a validation MSE of ≈ 0.62

(from the previous section). Ridge regularization reduced this error to ≈ 0.14, a substantial improvement. However, the simple degree 9 model from Part B.1 (MSE ≈ 0.077) still outperformed the regularized degree 12 model. This suggests that while regularization helps control overfitting, selecting the correct model complexity (degree) is often the most effective strategy.

Question 5: Perceptron

5.1 Weight Update

Given initial weight $w = [0, 5]$ and data point $x = [0, 1]$ (negative class, $y = -1$).

$$\text{Prediction: } \hat{y} = \text{sign}(w \cdot x) = \text{sign}([0, 5] \cdot [0, 1]) = \text{sign}(5) = +1$$

$$\text{True label is } -1 \implies \text{Misclassification}$$

$$w_{new} = w + yx = [0, 5] + (-1)[0, 1]$$

$$w_{new} = [0, 5] - [0, 1]$$

$$\Rightarrow \boxed{w_{new} = [0, 4]}$$

5.2 Convergence Iterations

The negative points (at distance 1) and positive points (at distance 2) are not linearly separable (they require a circular boundary).

$$\boxed{\infty}$$

5.3 Data Transformation

We need a transformation that separates points based on distance from the origin. The term $x^2 + y^2$ represents squared distance.

$$\boxed{[x, y] \to [x, y, x^2 + y^2]}$$

5.4 Possible Weight Vector

We seek weights for $\phi(x) = [x, y, x^2 + y^2, 1]$. Let $w = [0, 0, w_3, w_4]$. We need $w_3(r^2) + w_4 > 0$ for $r^2 = 4$ (positive) and $w_3(r^2) + w_4 < 0$ for $r^2 = 1$ (negative).

Let $w_3 = 1, w_4 = -2.5$:

Pos class: $1(4) - 2.5 = 1.5 > 0 \quad (\checkmark)$

Neg class: $1(1) - 2.5 = -1.5 < 0 \quad (\checkmark)$

$$\Rightarrow \boxed{w = [0, 0, 1, -2.5]}$$

5.5 Another Transformation

Using L1 norm (Manhattan distance) as the separating feature:

$$\boxed{[x, y] \to [x, y, |x| + |y|, 1]}$$

Question 6: Neural Network Representations

6.1 Full List of Representable Functions

- **Graph G_1 ($y = wx$):**
  - Represents: **(a)**.
- **Graph G_2 ($y = wx + b$):**
  - Represents: **(a), (b)**.
- **Graph G_3 ($y =$ ReLU$(wx + b)$):**
  - Represents: **(c), (d)**.
- **Graph G_4 ($y = w_2(w_1x + b_1) + b_2$):**
  - Equivalent to a single linear layer $y = Wx + B$.
  - Represents: **(a), (b)**.
- **Graph G_5 ($y = w_2$ReLU$(w_1x + b_1) + b_2$):**
  - Can scale, shift, and flip a ReLU function.
  - Represents: **(c), (d), (e)**.
- **Graph G_6 (Nested ReLUs):**
  - Can represent monotonic piecewise linear functions.
  - Represents: **(c), (d), (e), (f), (h)**.

- **Graph G_7 (Sum of two ReLUs):**
  - Universal approximator for piecewise linear functions with up to 2 kinks.
  - Represents: **(c), (d), (e), (f), (h)**.

6.2 Weights and Biases (Graphs $G_1 - G_5$)

Graph G_1:

$$\text{For (a) } 2x: \quad \boxed{w = 2}$$

Graph G_2:

$$\text{For (a) } 2x: \quad \boxed{w = 2, \quad b = 0}$$

$$\text{For (b) } 4x - 5: \quad \boxed{w = 4, \quad b = -5}$$

Graph G_3:

$$\text{For (c) ReLU}(2x - 5): \quad \boxed{w = 2, \quad b = -5}$$

$$\text{For (d) ReLU}(-2x - 5): \quad \boxed{w = -2, \quad b = -5}$$

Graph G_4 (Equivalent to $Wx+B$ where $W = w_1w_2, B = w_2b_1+b_2$):

$$\text{For (a) } 2x: \quad \text{Let } w_1 = 1, b_1 = 0$$

$$\implies \boxed{w_1 = 1, b_1 = 0, w_2 = 2, b_2 = 0}$$

$$\text{For (b) } 4x - 5: \quad \text{Let } w_1 = 1, b_1 = 0$$

$$\implies \boxed{w_1 = 1, b_1 = 0, w_2 = 4, b_2 = -5}$$

Graph G_5 ($y = w_2\textbf{ReLU}(w_1 x + b_1) + b_2$):

For (c) ReLU$(2x - 5)$: Set $b_2 = 0, w_2 = 1$

$$\implies \boxed{w_1 = 2, b_1 = -5, w_2 = 1, b_2 = 0}$$

For (d) ReLU$(-2x - 5)$: Set $b_2 = 0, w_2 = 1$

$$\implies \boxed{w_1 = -2, b_1 = -5, w_2 = 1, b_2 = 0}$$

For (e) $1 -$ ReLU$(x - 2)$:

$$\implies \boxed{w_1 = 1, b_1 = -2, w_2 = -1, b_2 = 1}$$

Question 7: Loss function

7.1 Equivalence of MLE and Cross-Entropy

We assume independent observations. The likelihood of the dataset is the product of individual probabilities. Given $y_k(x_n, w) = P(t_k = 1|x_n)$ and one-hot target vectors t_n:

$$L(w) = \prod_{n=1}^{N} \prod_{k=1}^{K} [y_k(x_n, w)]^{t_{kn}}$$

$$\xrightarrow{\ln(\cdot)} \ell(w) = \sum_{n=1}^{N} \sum_{k=1}^{K} t_{kn} \ln y_k(x_n, w)$$

Maximizing likelihood $\equiv$ Minimizing negative log-likelihood

$$E(w) = -\ell(w)$$

$$\Rightarrow \boxed{E(w) = -\sum_{n=1}^{N} \sum_{k=1}^{K} t_{kn} \ln y_k(x_n, w)}$$

7.2 Binary Classification with Noisy Labels

Let $y_n = P(\text{true class} = 1|x_n)$. The observed label $t \in \{0, 1\}$ is incorrect with probability ϵ. The effective probability of observing label 1 is:

$$\pi_n \triangleq P(t_n = 1|x_n) = y_n(1-\epsilon) + (1-y_n)\epsilon$$

$$\text{Likelihood: } L = \prod_{n=1}^{N} \pi_n^{t_n}(1-\pi_n)^{1-t_n}$$

$$\text{Negative Log-Likelihood: } E(w) = -\sum_{n=1}^{N} [t_n \ln \pi_n + (1-t_n)\ln(1-\pi_n)]$$

Substitute π_n :

$$\boxed{E(w) = -\sum_{n=1}^{N} \Big[t_n \ln\left((1-\epsilon)y_n + \epsilon(1-y_n)\right) + (1-t_n)\ln\left((1-\epsilon)(1-y_n) + \epsilon y_n\right)\Big]}$$

Verification if $\epsilon = 0$: $\quad \pi_n = y_n$

$$\Rightarrow E(w) = -\sum_{n=1}^{N} [t_n \ln y_n + (1-t_n)\ln(1-y_n)] \quad \checkmark$$

7.3 Error Function for Targets {-1, 1}

Target $t \in \{-1, 1\}$ and output $y \in [-1, 1]$. We map the network output to a probability $P(C_1|x)$ linearly:

$$P(t = 1|x) = \frac{1 + y}{2}, \quad P(t = -1|x) = \frac{1 - y}{2} \implies P(t|x) = \frac{1 + ty}{2}$$

$$\text{Likelihood: } L = \prod_{n=1}^{N} \left(\frac{1 + t_n y_n}{2} \right)$$

Error Function (Negative Log-Likelihood):

$$E(w) = -\sum_{n=1}^{N} \ln \left(\frac{1 + t_n y_n}{2} \right) = -\sum_{n=1}^{N} [\ln(1 + t_n y_n) - \ln 2]$$

Ignoring constant terms:

$$\Rightarrow \boxed{E(w) = -\sum_{n=1}^{N} \ln(1 + t_n y_n)}$$

Appropriate Activation Function: Since we need an output range of $[-1, 1]$, the suitable choice is:

$$\boxed{\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}}$$

Question 8: Stochastic Stability and the "Dead ReLU"

8.1 Distribution Proof

Let $z = w^\top x + b$. Since n is large, by the Central Limit Theorem (CLT), z approximates a Gaussian distribution. We compute its moments:

$$E[x_i] = 0, \quad E[w_i] = 0 \implies E[w_i x_i] = E[w_i]E[x_i] = 0$$

$$E[z] = \sum_{i=1}^{n} E[w_i x_i] + b = 0 + b = b$$

$$Var(w_i x_i) = E[(w_i x_i)^2] - (E[w_i x_i])^2 = E[w_i^2]E[x_i^2] - 0 = \sigma_w^2 \sigma_x^2$$

$$\sigma_z^2 = Var(z) = \sum_{i=1}^{n} Var(w_i x_i) = n\sigma_w^2 \sigma_x^2$$

$$\Rightarrow \boxed{z \sim \mathcal{N}(b, \sigma_z^2) \quad \text{where} \quad \sigma_z^2 = n\sigma_w^2 \sigma_x^2}$$

8.2 Activation Probability

The neuron is active if $a > 0 \iff z > 0$. Let $Z \sim \mathcal{N}(0, 1)$.

$$P(a > 0) = P(z > 0) = P\left(\frac{z - b}{\sigma_z} > \frac{0 - b}{\sigma_z}\right)$$

$$= P\left(Z > -\frac{b}{\sigma_z}\right) = 1 - \Phi\left(-\frac{b}{\sigma_z}\right)$$

Using symmetry $\Phi(-x) = 1 - \Phi(x)$:

$$\Rightarrow \boxed{P(a > 0) = \Phi\left(\frac{b}{\sigma_z}\right)}$$

8.3 The 3σ Threshold

Given $b = -3\sigma_z$:

$$P(a > 0) = \Phi\left(\frac{-3\sigma_z}{\sigma_z}\right) = \Phi(-3)$$

From standard normal table:

$$\Phi(-3) \approx 0.00135$$

Expected active neurons for $N = 1000$:

$$E[\text{active}] = 1000 \times 0.00135 = 1.35$$

$$\Rightarrow \boxed{\approx 1 \text{ neuron}}$$

8.4 The Zero-Gradient Proof

If $z_i \leq 0$ for all samples in the mini-batch, then $a_i = \max(0, z_i) = 0$. The derivative of ReLU is:

$$\frac{\partial a}{\partial z} = \mathbb{1}_{z>0} = 0 \quad (\text{since } z \leq 0)$$

$$\frac{\partial L}{\partial w} = \sum_{i \in \text{batch}} \frac{\partial L}{\partial a_i} \cdot \frac{\partial a_i}{\partial z_i} \cdot \frac{\partial z_i}{\partial w}$$

$$\frac{\partial L}{\partial w} = \sum_{i \in \text{batch}} \frac{\partial L}{\partial a_i} \cdot (0) \cdot x_i = 0$$

$$\Delta w = -\eta \frac{\partial L}{\partial w} = 0$$

$$\Rightarrow \boxed{\text{Weights do not update } (\Delta w = 0).}$$

Mathematical Implication: Since the gradient is zero, the weights and bias never change. The neuron remains permanently inactive ($z \leq 0$) for all future inputs. It effectively "dies" and reduces the network's capacity, as it can never be "resurrected" by gradient descent.

Question 9: Variance Preservation and Xavier Initialization

9.1 Symmetric Constraints

We consider a linear layer $y = Wx$. For a single output neuron $y_i = \sum_{j=1}^{n_{in}} W_{ij}x_j$:

$$Var(y_i) = Var\left(\sum_{j=1}^{n_{in}} W_{ij}x_j\right) \xlongequal{\text{independent sum}} \sum_{j=1}^{n_{in}} Var(W_{ij}x_j)$$

Using $Var(X) = E[X^2] - \underbrace{(E[X])^2}_{0}$ and $E[W] = E[x] = 0$:

$$Var(W_{ij}x_j) = E[W_{ij}^2]E[x_j^2] = Var(W)Var(x)$$

$$\Rightarrow Var(y_i) = \sum_{j=1}^{n_{in}} Var(W)Var(x) = n_{in} \cdot Var(W) \cdot Var(x)$$

$$\xrightarrow{\text{Require } Var(y)=Var(x)} \quad n_{in}Var(W) = 1 \quad \Rightarrow \quad \boxed{Var(W) = \frac{1}{n_{in}}}$$

Similarly, for the backward pass, gradients propagate via $W^\top$ (input dimension becomes n_{out}):

$$\xrightarrow{\text{Require } Var(\delta_{in})=Var(\delta_{out})} \quad n_{out}Var(W) = 1 \quad \Rightarrow \quad \boxed{Var(W) = \frac{1}{n_{out}}}$$

9.2 The Glorot Synthesis

We have conflicting constraints: Forward ($1/n_{in}$) and Backward ($1/n_{out}$). We compromise by satisfying the condition for the arithmetic mean of dimensions (n_{avg}).

$$\text{Constraints} \rightarrow \text{Compromise using } n_{avg} = \frac{n_{in} + n_{out}}{2}$$

$$\xrightarrow{\text{Require } n_{avg}Var(W)=1} \left(\frac{n_{in} + n_{out}}{2}\right) \cdot Var(W) = 1$$

$$\Rightarrow Var(W) = \frac{1}{\frac{n_{in}+n_{out}}{2}} \quad \Rightarrow \quad \boxed{Var(W) = \frac{2}{n_{in} + n_{out}}}$$

9.3 Depth-Induced Variance

Let the layer gain be $G = n \cdot Var(W)$. The variance propagates as $Var(y^{(l)}) = G \cdot Var(y^{(l-1)})$.

$$\text{Layer 1: } Var(y^{(1)}) = G \cdot Var(x^{(0)})$$

$$\xrightarrow{\text{Layer 2}} Var(y^{(2)}) = G \cdot Var(y^{(1)}) = G(G \cdot Var(x^{(0)})) = G^2 Var(x^{(0)})$$

$$\vdots$$

$$\xrightarrow{\text{Induction to Layer } L} \boxed{Var(y^{(L)}) = G^L \cdot Var(x^{(0)})}$$

9.4 The Stability Bound

We are given the initialization condition $n \cdot Var(W) = 1 + \epsilon$, which implies the gain is $G = 1 + \epsilon$.

$$\text{From previous part: } Var(y^{(L)}) = G^L \cdot Var(x^{(0)})$$

$$\xrightarrow{\text{Substitute } G=1+\epsilon} Var(y^{(L)}) = (1+\epsilon)^L \cdot Var(x^{(0)})$$

$$\xrightarrow{\text{Approx } 1+\epsilon \approx e^{\epsilon}} \boxed{Var(y^{(L)}) \approx e^{\epsilon L} \cdot Var(x^{(0)})}$$

This implies **exponential growth** of variance with depth L.

9.5 Numerical Analysis

Given $L = 150$ and $\epsilon = 0.02$. We compute the amplification factor $A = (1+\epsilon)^L$.

$$A = (1.02)^{150} \xrightarrow{\ln(\cdot)} \ln A = 150 \ln(1.02)$$

$$\xrightarrow{\ln(1+x) \approx x} \ln A \approx 150 \times 0.02 = 3$$

$$\xrightarrow{\exp(\cdot)} A \approx e^3 \approx 20.08$$

$$\Rightarrow \boxed{\text{Amplification Factor} \approx 20}$$

Risk for FP16: FP16 has a max value of ≈ 65504. An amplification of $\times 20$ in signal (and potentially gradients) in a deep net ($L = 150$) can easily cause values to explode to Infinity (NaN) or vanish, leading to training instability.

References

1. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. (Reference for Polynomial Regression, Maximum Likelihood Estimation, Cross-Entropy Loss, and the theoretical foundation of Neural Networks).

2. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. (Reference for Computation Graphs, ReLU activation dynamics, "Dead ReLU" phenomenon, and optimization stability).

3. Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics* (pp. 249-256). (Primary source for the "Glorot Synthesis" and Xavier Initialization discussed in Question 9).

4. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer. (Reference for Bias-Variance trade-off and Ridge Regression analysis).